Gen AI 시대의 파이썬: 데이터 분석 기초 및 전처리

Numpy와 Pandas를 내 전속 '데이터 분석가'로 채용하는 법

이동현

2026년 3월 4일

한국외국어대학교 Social Science & AI융합학부

세션 시간표

세션	시간	주요 내용
1	09:30 – 10:20	2주차 복습 및 데이터 분석 환경(Numpy, Pandas) 셋업
2	10:30 – 11:20	Numpy 기초: 다차원 배열(Array) 생성과 브로드캐스팅
3	11:30 – 12:20	Pandas 핵심 1: DataFrame 구조 이해 및 타이타닉 데이터 로드
4	12:30 – 13:20	Pandas 핵심 2: 타이타닉 생존자 인덱싱, 슬라이싱, 조건부 필터링
	13:20 – 14:20	**점심 시간**
5	14:20 – 15:10	데이터 전처리 1: 결측치(NaN) 탐지, 대체 및 이상치 제거
6	15:20 – 16:10	데이터 전처리 2: 데이터 병합(Merge/Concat) 및 그룹화(Groupby)
7	16:20 – 17:10	[프로젝트 파트 1] 에어코리아 확정데이터 로드 및 결측치 처리
8	17:20 – 18:10	[프로젝트 파트 2] 미세먼지 데이터 기술통계, T-test 및 회귀분석 예측

세션 1 & 2: NumPy 핵심 (배열과 브로드캐스팅)

왜 NumPy를 사용하는가?

- **데이터 과학의 기반:** Pandas, Matplotlib, Scikit-learn 등 모든 라이브러리의 근간
- **압도적인 성능:** 파이썬 표준 리스트보다 훨씬 빠른 연산 속도와 낮은 메모리 사용량
- **벡터화(Vectorization):** 루프(for문) 없이 배열 전체에 대한 연산을 한 번에 처리
- **C / Fortran 기반:** 내부적으로 최적화된 저수준 언어로 동작하여 인터프리터 오버헤드 최소화

핵심 철학
"NumPy 배열을 대상으로 `for` 반복문을 작성하고 있다면, 아마도 잘못된 방식으로 접근하고 있을 가능성이 높다."

압도적인 성능의 비결: 벡터화와 메모리 구조

Python 리스트

- 흩어져 있는 객체에 대한 포인터 배열
- 타입 체크 오버헤드 발생

NumPy ndarray

- **연속된 메모리 블록**에 동일 타입 데이터 저장
- 캐시 효율성 극대화 및 하드웨어 가속 활용

구분	Python 리스트	NumPy 배열
메모리 구조	객체 포인터 (비연속)	실제 값 (연속)
연산 방식	요소별 반복 처리	C 레벨에서 일괄 처리
속도	느림 (순차적)	**매우 빠름 (병렬적)**

기본 배열 생성: np.array

- **ndarray (N-dimensional Array):** NumPy의 핵심 객체
- **제약:** 동일한 데이터 타입(Homogeneity)만 저장 가능→ 빠른 연산의 비결

```
import numpy as np

# 1. Using Python List
arr1d = np.array([1, 2, 3, 4, 5])
arr2d = np.array([[1, 2, 3], [4, 5, 6]])

# 2. Specifying dtype
arr_float = np.array([1, 2, 3], dtype='float32') # Force float type
```

[참고] np라는 별칭(alias)은 전 세계적인 관례입니다.

특정 값으로 배열 생성: 플레이스홀더(Placeholder)

- **필요성:** 데이터가 들어올 공간을 미리 확보하여 메모리 효율성 증대

```
# All zeros / ones
zeros = np.zeros((3, 4))   # 3x4 filled with 0.0
ones = np.ones((2, 3))     # 2x3 filled with 1.0

# Specific value / Identity matrix
full = np.full((2, 2), 99) # 2x2 filled with 99
eye = np.eye(3)            # 3x3 Identity Matrix (diag=1)
```

순차 및 로그 스케일 데이터 생성

- **arange:** 특정 간격(step) 기준 (Python range와 유사)
- **linspace:** 특정 개수(num) 기준 (균등 분할)
- **logspace:** 로그 스케일 상의 균등 분할

```
# 0 to 10 (exclusive) with step 2
seq = np.arange(0, 10, 2)     # [0, 2, 4, 6, 8]

# 0 to 1 with 5 points (inclusive)
grid = np.linspace(0, 1, 5)   # [0. , 0.25, 0.5 , 0.75, 1. ]

# Log scale: 10^0 to 10^2 with 3 points
logs = np.logspace(0, 2, 3)   # [1. , 10. , 100. ]
```

난수 기반 배열 생성 (np.random)

- **rand:** [0, 1) 범위 균등 분포
- **randn:** 평균 0, 표준편차 1의 표준 정규 분포 (종 모양)
- **randint:** 지정된 범위 내 정수 난수

```
# Uniform dist (3x2 matrix)
u = np.random.rand(3, 2)

# Normal dist (shape=(1000,))
n = np.random.randn(1000)

# Random integers (1 to 100, size 5)
i = np.random.randint(1, 101, size=5)
```

[AI 활용] 실무 데이터 셋업 프롬프트

실무 시나리오: 가상 주문 데이터 생성
시뮬레이션 분석을 위해 1,000명의 고객에 대해 각각 1 10 사이의 임의의 주문 수량을 생성해야 합니다.

AI 프롬프트 예시

> "*Numpy*를 사용해서 *1,000*개의 정수를 랜덤하게 생성할거야. 범위는 *1*부터 *10*까지고(*10* 포함), 일관된 결과를 위해 시드(*Seed*)를 *42*로 설정해 줘. 그리고 상위 *5*개 데이터와 배열의 형태를 출력하는 코드를 작성해 줘."

→ **정석 코드를 배운 뒤, 복잡한 시나리오를 AI에게 맡기면 훨씬 효율적입니다.**

배열의 속성(Attribute) 이해

- 데이터 과학 오류의 대부분은 **배열의 형상(Shape) 불일치**에서 발생합니다.

```
arr = np.random.randint(1, 10, size=(3, 4))

print(arr.ndim)   # Number of axes (Dimensions): 2
print(arr.shape)  # Shape: (3, 4)
print(arr.size)   # Total elements: 12
print(arr.dtype)  # Data type: int64 (or int32)
print(arr.nbytes) # Memory size in bytes
```

인덱싱과 슬라이싱: 데이터 접근 원리

- **인덱싱:** 특정 위치의 단일 요소 선택 (`arr[row, col]`)
- **슬라이싱:** 범위 선택 (`start:stop:step`)
- **다차원 접근:** 쉼표(,)로 축 구분 가능. `arr[1][2]` 보다 `arr[1, 2]`가 효율적입니다(단일 연산).

1	2	3
4	5	6

Row 1

`arr[1, 1]` → 5

[코드] 인덱싱 및 슬라이싱 실습

```
# 3x3 Matrix representing customer scores
scores = np.array([[80, 85, 90],
                   [70, 75, 80],
                   [60, 65, 70]])

# 1. Row selection
first_row = scores[0, :]       # [80, 85, 90]

# 2. Sub-matrix selection
subset = scores[:2, 1:]        # Rows 0-1, columns 1-2
# [[85, 90],
#  [75, 80]]

# 3. Last column selection
last_col = scores[:, -1]       # [90, 80, 70]
```

[주의] 뷰(View) vs 복사(Copy)

- **뷰 (View):** 슬라이싱 결과는 원본 메모리를 공유하는 '창'입니다. 수정 시 원본이 바뀝니다.
- **복사 (Copy):** 원본과 독립적인 데이터가 필요하면 `.copy()`를 명시해야 합니다.

```
orig = np.arange(5)
view = orig[1:4]
view[0] = 999
print(orig) # [0, 999, 2, 3, 4] -> Original corrupted!

# Safe Way
safe_copy = orig[1:4].copy()
safe_copy[1] = 0 # orig stays same
```

불리언 인덱싱: 조건부 데이터 추출

- SQL의 `WHERE` 절과 유사한 강력한 기능
- 배열에 비교 연산을 적용하여 **마스크(Mask)** 생성
- 다중 조건 결합 시 &(AND), |(OR) 사용 (괄호 필수!)

```
# Sales data
sales = np.array([400, 700, 200, 800, 100])

# 1. Create mask
mask = sales > 500 # [False, True, False, True, False]

# 2. Filter
print(sales[mask])  # [700, 800]

# 3. Direct filtering & Assignment
sales[sales < 300] = 0 # Replace outliers with 0
```

[AI 활용] 복합 조건 필터링 프롬프트

실무 시나리오: 이상치 정제

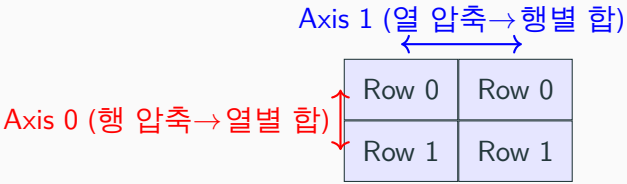
이커머스 데이터에서 결제 금액이 0원 이하이거나, 평균 구매 금액의 3배를 초과하는 '특이 데이터'를 찾아야 합니다.

AI 프롬프트 예시

> "*Numpy* 배열 $prices$가 있어. *1*) 값이 *0* 이하인 인덱스를 찾아서 *0*으로 바꾸고, *2*) 값이 전체 평균 $+$ *3**표준편차를 넘는 데이터를 필터링해서 $outliers$ 변수에 저장하는 코드를 작성해 줘."

배열의 연산: ufuncs와 Axis

- **ufuncs (Universal Functions):** 요소별 산술 연산 최적화 (`+`, `-`, `*`, `/`, `np.sqrt`, etc.)
- **집계 함수:** 배열의 전체 또는 특정 축(**axis**)을 따라 요약
- **Axis 개념:** 연산 후 **"압축되어 사라지는 차원"**으로 이해



브로드캐스팅(Broadcasting)의 규칙

- **정의:** 형상이 다른 배열 간의 연산을 가능하게 하는 메커니즘
- **작동 조건:**
  1. 뒤에서부터 비교했을 때 차원의 크기가 동일하거나
  2. 둘 중 하나의 차원 크기가 1인 경우
- **이점:** 실제 메모리 복사 없이 가상으로 확장하여 연산 → 메모리 효율적

브로드캐스팅 시각화

shape (3, 3)

10	20	30
40	50	60
70	80	90

+

shape (3,)

1	2	3
1	2	3
1	2	3

=

Result (3, 3)

11	22	33
41	52	63
71	82	93

1차원 벡터가 행렬의 **모든 행에 가상으로 복제**되어 연산됨

배열의 변환 및 결합

- **reshape:** 물리적 데이터는 그대로 둔 채 형태만 변경 (주로 뷰 반환)
- **ravel vs flatten:** 1차원으로 풀기 (ravel은 뷰 우선, flatten은 복사)
- **concatenate:** 배열 합치기 (vstack, hstack 활용 가능)

```
arr = np.arange(6) # [0, 1, 2, 3, 4, 5]

# Reshape 1D to 2D
mat = arr.reshape(2, 3)

# Combine
a = np.array([1, 2])
b = np.array([3, 4])
merged = np.vstack((a, b)) # [[1, 2], [3, 4]]
```

[중요] 원소 추가/삭제의 비용

- **`np.append`, `np.delete`:** NumPy 배열은 메모리 크기가 고정되어 있어, 추가/삭제 시마다 **배열 전체를 새로 복사**합니다.

성능 팁
반복적으로 데이터를 추가해야 한다면, 먼저 Python **리스트(List)**에 담은 뒤 마지막에 `np.array()`로 한 번만 변환하세요.

```
# Bad logic (Slow)
arr = np.array([])
for i in range(100):
   arr = np.append(arr, i)

# Good logic (Fast)
list_data = []
for i in range(100):
   list_data.append(i)
arr = np.array(list_data)
```

세션 3 & 4: Pandas 데이터 제어의 핵심

Pandas: 파이썬의 고성능 데이터 분석 도구

- **정의:** 웨스 맥키니(Wes McKinney)가 개발한 파이썬 데이터 구조 및 분석용 라이브러리
- **특징:** 엑셀보다 훨씬 큰 대용량 데이터(빅데이터)를 빠르고 유연하게 처리
- **역할:** SQL, Excel의 기능을 파이썬 환경으로 가져와 "자동화" 가능
- **자료구조:**
  - **Series:** 1차원 배열 (엑셀의 한 열)
  - **DataFrame:** 2차원 표 (엑셀 시트 전체)

```
# Installation & Import
# pip install pandas
import pandas as pd
```

Pandas의 핵심 자료구조: Series와 DataFrame

Series (1D)

- index-value 쌍의 구조
- 단일 데이터 타입 권장

DataFrame (2D)

- 여러 개의 Series가 결합된 형태
- 행(Row)과 열(Column) 인덱스 존재

Columns (Axis 1)

Index (Axis 0)

	col1	col2	col3
0	val	val	val
1	val	val	val

[코드] Pandas 객체 생성하기

```python
# 1. From List (Series)
s = pd.Series([100, 200, 300], index=['A', 'B', 'C'])

# 2. From Dictionary (DataFrame)
data = {
    'item': ['Apple', 'Banana', 'Cherry'],
    'price': [2500, 3000, 1500],
    'qty': [10, 5, 20]
}
df = pd.DataFrame(data)

# 3. From NumPy Array
arr = np.random.rand(3, 2)
df_np = pd.DataFrame(arr, columns=['val1', 'val2'])
```

데이터 로드 가이드 (내장 데이터 vs 외부 파일)

- **sns.load_dataset():** 학습용 데이터를 패키지에서 바로 불러오기
- **read_csv:** 실제 현업의 CSV 파일을 데이터프레임으로 호출

```
import pandas as pd
import seaborn as sns # for loading dataset

# 1. Load built-in dataset (For today's practice)
df = sns.load_dataset('titanic')

# 2. Load from external file (For reference)
# df_csv = pd.read_csv('data.csv', encoding='utf-8-sig') # Anti-break encoding
# df_huge = pd.read_csv('huge.csv', usecols=['survived', 'age']) # Partial load
```

[AI 활용] 내장 데이터 로드 및 초기 탐색 프롬프트

실무 시나리오: 타이타닉 생존자 데이터 분석 방향 수립
유명한 기계학습 예제인 타이타닉 호 데이터를 파이썬에서 불러오고, 어떤 정보들이 결측되어 있는지 빠르게 파악해야 합니다.

AI 프롬프트 예시

> "`seaborn` 패키지를 사용해서 'titanic' 데이터셋을 Pandas 데이터프레임으로 불러와 줘. 그리고 데이터의 전체 행/열 개수를 확인하고, 위에서부터 5개의 행을 출력해서 데이터가 어떻게 생겼는지 보여주는 파이썬 코드를 작성해 줘."

데이터 확인 및 탐색 필수 5종

- **`head()` / `tail()`:** 상하위 데이터 일부 확인 (기본 5개)
- **`shape`:** 행과 열의 개수 확인 (`(row, col)` 튜플)
- **`info()`:** 데이터 타입(Dtype), 누락 데이터(Non-Null) 현황 파악
- **`describe()`:** 수치형 데이터의 주요 기술 통계량 (평균, 표준편차, 4분위수)

```
df = sns.load_dataset('titanic')
print(df.head(5))       # Top 5 rows
print(df.info())        # Structural overview (Check missing 'age', 'deck')
print(df.describe())    # Statistical overview (Average 'fare', 'age')
```

범주형 데이터 분석 정석

- **unique()**: 중복을 제외한 고유값 목록
- **value_counts()**: 각 항목별 빈도수 (가장 많이 쓰임!)

```
# Which classes exist?
print(df['class'].unique())         # ['Third', 'First', 'Second']

# Count of passengers per class
print(df['class'].value_counts())

# Normalized (Survival rate placeholder)
print(df['survived'].value_counts(normalize=True)) # 0: Dead, 1: Survived
```

데이터 선택의 정석: loc vs iloc

구분	loc (Location)	iloc (Integer-location)
기준	**이름(Label)** 기준	**순서(Position)** 기준
슬라이싱	끝점(endpoint) **포함**	끝점 **제외** (파이썬 기본과 동일)
용도	"회원ID가 U101인 데이터"	"데이터의 3번째 행"

주의사항

`df['column_name']`은 Series를 반환하며, `df[['col1', 'col2']]`는 DataFrame을 반환합니다.

[코드] 데이터 선택 실습

```python
# 1. Label-based selection
# Get row with index 0 and column 'fare'
val = df.loc[0, 'fare']

# 2. Slicing with labels (includes index 2)
subset = df.loc[0:2, 'survived':'age']

# 3. Position-based selection
# Get first 2 rows and columns 0 and 1
subset_p = df.iloc[:2, :2]

# 4. Multi-column selection
features = df[['pclass', 'sex', 'age']]

# 5. Row Slicing (Orthodox)
# Includes index 0, 1, 2
rows = df[0:3]
```

불리언 마스킹: Pandas 필터링의 핵심

- **원리:** 조건식의 결과로 나오는 True/False 마스크를 인덱서로 활용
- **다중 연산자:** & (AND), | (OR), ~ (NOT)
- **문법 보너스:** 각 조건은 반드시 **괄호()**로 묶어야 연산 우선순위 오류를 피할 수 있습니다.

```
# Female passengers in 1st class
mask = (df['sex'] == 'female') & (df['pclass'] == 1)
first_class_females = df[mask]

# Using helper method
high_fares = df[df['fare'].between(100, 500)]
```

[AI 활용] 비즈니스 조건 필터링 프롬프트

실무 시나리오: 특정 고객군 생존율 탐색

'여성과 아이들 먼저(Women and children first)'라는 해상 조난 대피 규칙이 타이타닉호에서도 실제로 지켜졌는지 확인해보고자 합니다.

AI 프롬프트 예시

> "*Pandas*를 이용해 타이타닉 데이터에서 승객을 필터링할 거야. *1*) 나이(`age`)가 *10*살 미만이거나, *2*) 성별(`sex`)이 여성('*female*')인 승객들만 추출해 줘. 그리고 이 추출된 그룹의 요약 통계를 `describe()`로 보여줘서 전반적인 생존율(`survived` 평균)을 확인할 수 있게 코드를 작성해 줘."

세션 5 & 6: 데이터 전처리 (결측치, 병합, 그룹화)

데이터 품질 관리가 분석의 80% 입니다

GIGO: Garbage In, Garbage Out
데이터가 깨끗하지 않으면 아무리 뛰어난 AI 모델도 잘못된 통찰을 내놓습니다.

- **결측치(Missing Values):** 값이 비어 있는 경우 (`NaN`, `None`)
- **중복치(Duplicates):** 동일한 기록이 여러 번 저장된 경우
- **이상치(Outliers):** 상식적인 범위를 크게 벗어난 값 (전산 오류 등)
- **부적절한 타입:** 숫자가 문자열로 저장된 경우 (`'1,000'` $\rightarrow$ `1000`)

결측치 탐지: 데이터의 '구멍' 찾기

- **isnull():** 데이터가 비어 있는지 확인 (T/F 반환)
- **sum():** 컬럼별 빈 칸의 개수 합계

```
import pandas as pd

# 1. Total missing values per column
print(df.isnull().sum())

# 2. Percentage of missing values
print(df.isnull().mean() * 100)

# 3. Quick visual check (sample contains NaN?)
print(df[df['age'].isnull()].head())
```

결측 처리의 정석: 삭제 vs 대체

처리 방법	장점	단점 / 주의사항
삭제 (Drop)	데이터 오염 원천 차단	중요한 정보 손실 가능성
대체 (Fill)	데이터 손실 없음	분석 결과의 편향(Bias) 발생 가능

대체값 선정 기준

- **수치형:** 평균(mean), 중앙값(median)
- **범주형:** 최빈값(mode), 'Unknown' 등 특정 문자열
- **시간적:** 전/후 데이터로 채우기 (ffill, bfill)

[코드] 결측치 처리 실습

```python
# 1. Drop rows with missing 'user_id'
df.dropna(subset=['user_id'], inplace=True)

# 2. Fill missing ages with MEDIAN (Robust to outliers)
median_age = df['age'].median()
df['age'].fillna(median_age, inplace=True)

# 3. Fill categorical NaN with fixed string
df['rating'].fillna('Bronze', inplace=True)

# 4. Forward fill (Copy previous non-NaN value)
df['price'].fillna(method='ffill', inplace=True)
```

데이터 형 변환 (Type Conversion)

- **astype()**: 기존 컬럼을 원하는 자료형으로 강제 변환
- **pd.to_numeric()**: 숫자가 섞인 문자열을 숫자로 변환 (에러 처리 가능)

```
# 1. Convert to Int/Float
df['qty'] = df['qty'].astype(int)

# 2. Convert to Category (Memory efficient)
df['rating'] = df['rating'].astype('category')

# 3. Safe conversion with error handling
df['price'] = pd.to_numeric(df['price'], errors='coerce')
```

[AI 활용] 데이터 클리닝 파이프라인 프롬프트

실무 시나리오: 복합 데이터 결함 해결

이커머스 고객 데이터에서 1) '나이' 결측치는 평균으로 채우고, 2) '이름'이 없는 고객은 삭제하고, 3) '주소'가 없는 경우 '미기입'으로 통일해야 합니다.

AI 프롬프트 예시

> "*Pandas* 데이터프레임 `df`에 대한 정제 코드를 짜줘. `'age'` 열의 결측치는 평균값으로, `'address'` 열은 '미기입'으로 채워줘. `'name'` 열이 비어 있는 행은 과감하게 삭제해 줘. 모든 변경사항은 `inplace=True`로 적용해 줘."

데이터 병합: Merge (VLOOKUP 그 이상)

- **Inner Join:** 두 테이블 모두에 키가 존재하는 경우만 (교집합)
- **Left Join:** 왼쪽 테이블(기준)은 모두 보존, 오른쪽은 매칭되는 것만 (SQL 표준)
- **Outer Join:** 양쪽 테이블의 모든 데이터를 합침 (합집합)

Users

ID	Name
U1	Alice
U2	Bob

Orders

ID	Price
U1	100
U3	200

Inner Join: Only U1 matches
Left Join: U1 and U2 (U2 gets NaN for Price)

[코드] 병합(Merge)과 결합(Concat)

```python
# 1. Merge (JOIN by key)
# users: [user_id, name] / orders: [user_id, amount]
merged = pd.merge(users, orders, on='user_id', how='left')

# 2. Concat (Stacking by axis)
df1 = pd.DataFrame({'val': [1, 2]})
df2 = pd.DataFrame({'val': [3, 4]})

# Vertical stack (Rows)
vertical = pd.concat([df1, df2], axis=0)

# Horizontal stack (Columns)
horizontal = pd.concat([df1, df2], axis=1)
```

데이터 요약의 정석: GroupBy (분할-적용-결합)

1. **Split:** 특정 기준(예: 등급)에 따라 데이터를 그룹으로 분할
2. **Apply:** 각 그룹별로 통계 함수(합계, 평균 등)를 적용
3. **Combine:** 결과를 하나의 요약 테이블로 결합

```
# Rating-wise payment stats
# [Rating] (Index) | [Payment] (Value)
df.groupby('rating')['payment'].sum()

# Multi-aggregate
df.groupby('rating').agg({'payment': 'sum', 'age': 'mean'})
```

[AI 활용] 다차원 그룹 집계 프롬프트

실무 시나리오: 마케팅 성과 대시보드 요약

전체 주문 데이터를 회원 등급별로 그룹화하여 1) 총 매출액, 2) 평균 구매 단가, 3) 주문 건수를 한 번에 계산해야 합니다.

AI 프롬프트 예시

> "데이터프레임 `df`를 `'rating'` 열로 그룹화해 줘. 그리고 `'payment'` 열에 대해서는 합계(*sum*), 평균(*mean*), 개수(*count*)를 동시에 구해서 보여주는 코드를 작성해 줘. 결과 테이블의 컬럼 이름도 알기 쉽게 바꿔줘."

수치형 데이터의 관계: 상관분석(Correlation)

- **corr():** 변수 간 선형적 관계의 강도를 지수화 (-1 +1)
- **해석:** 1에 가까울수록 양의 상관관계, -1에 가까울수록 음의 상관관계

```
# Correlation matrix for numerical columns
corr_matrix = df.corr()
print(corr_matrix['payment'].sort_values(ascending=False))
```

Pandas 내장 시각화 (Quick EDA)

- **plot():** 별도 라이브러리 없이 즉석에서 그래프 생성
- **kind 옵션:** bar, hist, box, scatter, pie

```
# 1. Bar Chart (Categorical)
df['rating'].value_counts().plot(kind='bar')

# 2. Histogram (Distribution)
df['age'].plot(kind='hist', bins=20)

# 3. Scatter Plot (Relationship)
df.plot(kind='scatter', x='age', y='payment')
```

세션 7 & 8: [미니 프로젝트] 데이터 마트 구축

프로젝트 시나리오: 대용량 공공데이터 정제

- **상황:** 에어코리아 홈페이지에서 다운받은 대용량 미세먼지 확정데이터를 분석용으로 정제합니다.
- **준비물:** `airkorea_2023.csv` (2023년 측정소별 시간대별 미세먼지 관측 데이터)
- **Mission:**
  1. 데이터 로드 및 결측치/이상치 현황 파악
  2. 통계적 오류 방지를 위한 이상치(음수 등) 제거
  3. 결측값(NaN)을 전체 평균(Mean)으로 대체 보간
  4. 월별(Month) 파생변수 생성
  5. 요약: 특정 지역(예: 서울)의 월별 미세먼지(PM10) 농도 평균 추이 도출

[정석] 프로젝트 1단계: 로드 및 결측/이상치 탐지

```python
import pandas as pd
import numpy as np

# 1. Loading
df = pd.read_csv('airkorea_2023.csv', encoding='utf-8-sig')

# 2. Exploration
print(df.info()) # Check missing values in 'pm10' column
print(df.describe()) # Check min values (negative means error!)

# 3. Outlier Handling: Replace negative PM10 values with NaN
df.loc[df['pm10'] < 0, 'pm10'] = np.nan
```

[정석] 프로젝트 2단계: 결측치 보간 및 그룹 요약

```python
# 4. Missing Values: Fill with overall PM10 mean
pm10_mean = df['pm10'].mean()
df['pm10'].fillna(pm10_mean, inplace=True)

# 5. Extract specific region (e.g., Seoul)
seoul_df = df[df['sidoName'] == 'Seoul']

# 6. Derived Variable: Extract 'Month' from 'date' string (YYYY-MM-DD)
seoul_df['month'] = seoul_df['date'].str[5:7]

# 7. Final Summary: Monthly Average PM10 in Seoul
monthly_trend = seoul_df.groupby('month')['pm10'].mean()
print(monthly_trend)
```

[AI 활용] 프로젝트 고도화 (1): 에어코리아 데이터 전처리와 기술통계

학습 확장: 대규모 공공데이터 정제

앞서 배운 개념을 총동원해, 에어코리아 확정데이터(CSV 여러 개)를 병합하고 정제해 봅시다.

AI 프롬프트 예시

> "*Numpy*와 *Pandas*를 사용해서 에어코리아 *2023*년 측정소별 확정데이터(*CSV* 파일들)를 한 번에 읽어서 하나로 합쳐줘. 측정 이상치(음수 등)를 제거하고, 결측치는 전체 평균으로 대체해 줘. 마지막으로 `describe()` 함수를 사용해 전체 지역의 미세먼지(*PM10*) 농도에 대한 기술 통계를 출력하는 코드를 짜줘."

[AI 활용] 프로젝트 고도화 (2): 통계적 가설 검정과 예측 시각화

데이터 머신러닝/통계분석으로의 확장

단순 요약을 넘어, 그룹 간 유의미한 차이가 있는지 검증(T-test)하고 미래 추이를 예측(Regression)합니다.

AI 프롬프트 예시

> "정제된 데이터를 바탕으로 서울과 부산의 미세먼지(*PM10*) 연평균 농도에 통계적으로 유의미한 차이가 있는지 독립표본 *T-test*(*SciPy*)를 수행해 줘. 그리고 월별 미세먼지 변화 추이를 보여주는 산점도(*Scatter plot*)를 그리고, 그 위에 선형 회귀분석(*Linear Regression*) 추세선을 함께 시각화하는 코드를 작성해 줘."

[실전 과제] 나만의 대기질 분석 파이프라인 만들기

자유도 높은 2시간 미니 프로젝트: 데이터저널리즘 실습
단순한 통계 처리를 넘어, 미세먼지 데이터 속에서 **새로운 인사이트**를 도출하고 '기사 제목'을 뽑아봅시다.

과제 예시 (아래 중 택 1 또는 완전 자유 주제)

- **정책 효과성 검증:** "노후 경유차 제한 정책 시행 전후, 도심(서울)과 외곽 지역의 초미세먼지(PM2.5) 감소 폭은 달랐을까?"
- **생활 밀착형 분석:** "직장인 출퇴근 시간(08시, 19시)과 주말 오후 중 실제로 공기가 더 탁한 시간/요일은 언제일까?"
- **이상 기후 탐지:** "여름 장마철에도 미세먼지가 나쁨 수준이었던 이례적인 날들은 언제였으며, 그 원인(황사 등)은 무엇일까?"

제출물: 1. 데이터 정제 과정 + 2. 분석 코드 + 3. **데이터로 새롭게 알게 된 사실(한 줄 요약)**이 포함된 노트북 파일

마무리 및 회고

- **정석이 먼저입니다:** AI는 도구일 뿐, 데이터의 흐름과 정제 원리를 이해하는 것이 분석가의 핵심 역량입니다.
- **Pipeline Thinking:** 데이터를 읽고, 씻어내고, 합치고, 요약하는 '공정'을 설계하세요.
- **다음 예고:** 완성된 분석 마트를 바탕으로 '눈에 보이는 인사이트', **시각화 (Visualization)**를 배웁니다.

데이터 분석의 첫걸음, 수고하셨습니다!